ATML Project (VLMs and Counting Shapes) - Study 3

Methodology notes and predictions, written before any model runs

> **Main hypothesis:** Unnamed clutter (squiggles, blobs, etc.) should hurt counting less than nameable clutter (wrong-colour shapes, look-alikes, similar objects), because the model already ignores what it can't name. Not all clutter is equal -> a VLM is distracted more by things it can name than by things it can't name.

General setup (same for every hypothesis)

- **Model:** Qwen3-VL-2B-Instruct (same model as the original paper and our studies 1 and 2), pinned version, 16-bit, greedy answers (no randomness), same question for every version of a scene.
- **Data:** Mohid's 4 datasets (Shapes, Shapes Hard, Objects, Lines). Within a scene, the targets are pixel-identical across all versions -> all versions have the same correct answer, so any accuracy drop comes from the clutter.
- **Accuracy:** counting datasets -> "exact" (all 4 counts right). Lines -> "acc" (right box).
- **Acc drop** = Acc on clean pictures - Acc on cluttered pictures, using the same scenes.
- **95% interval:** bootstrap over scenes (10,000 resamples).
- All predictions are written down before running the model. Every result gets reported, including ones that go against us.

Stage 0: Floor check (before the main run)

- Run a small trial (20 images per version) and check clean accuracy.
- If clean accuracy is below 0.30 (counting) or 0.40 (Lines), the model is too weak for that dataset -> clutter can't make a near-zero score visibly worse. That dataset then switches to Qwen3-VL-8B.
- Also check that almost all answers can be read by the scorer (the "parsed" column).

H1: Nameable clutter hurts more than clutter with no name

Stage H1.1

- Measure the accuracy drop for each type of clutter. Acc drop = Acc on clean pictures - Acc on cluttered pictures, using the same scenes. The bigger the Acc drop, the more that clutter hurt the VLM.

Stage H1.2 -> Shapes dataset

- Compare nameable clutter with unnamed clutter:

```
Acc drop(lookalikes) - avg( Acc drop(squiggles), Acc drop(blobs) )
```

Stage H1.3 -> Shapes Hard dataset

- Same as Stage H1.2, but with more groups:

```
avg( Acc drop(wrong_color), Acc drop(lookalikes), Acc drop(text_traps) )
- avg( Acc drop(squiggles_2), Acc drop(squiggles_4),
      Acc drop(squiggles_8), Acc drop(squiggles_16) )
```

To evaluate: We calculate a 95% interval for the difference -> the range in which the true value likely falls, given how noisy the results are.

- Whole range above 0 in both datasets -> nameable clutter hurts more (supports our hypothesis).
- Whole range below 0 in both datasets -> unnamed clutter hurts more (opposite of our hypothesis).
- Range includes 0 -> no clear difference.

H2: Do more complex squiggles hurt counting more, or less? -> Shapes Hard

When the model counts shapes, does a more complicated squiggle nearby distract it more, or does the model ignore it more because it looks like nothing it has a name for?

Idea	Reasoning	Prediction
"Busy clutter"	More curves make the picture busier, so the model gets more distracted.	Accuracy goes **down** as squiggles get more complex.
"Names"	The more complicated a squiggle is, the harder it is to name, so the model ignores it more.	Accuracy stays the **same or goes up** as squiggles get more complex.

Stage H2.1

- Run the model on all 4 squiggle versions (2, 4, 8, 16 curves) + the clean version and measure accuracy for each version. Acc = how often the model gets all 4 counts right.

Stage H2.2: Nameability score

- Show every squiggle on its own to the model and ask 5 times: "*What does this shape look like? Answer in 1 word.*"
- Same word every time -> easy to name -> high nameability. Different words each time -> hard to name -> low nameability.
- Nameability score = (number of times the most common word appears - 1) / 4 -> 0 (all 5 different) to 1 (always the same word).
- Each image gets the average nameability score of the squiggles in it.
- Since the answers need to vary, these 5 answers are sampled with some randomness (not greedy). The settings are fixed before the run.

Stage H2.3: Two analyses (regression, separating nameability and complexity)

- **Main:** how accuracy changes each time the number of curves doubles (2 -> 4 -> 8 -> 16). This is the slope (each doubling = 1 equal step).
- **Untangling:** regression of "counted correctly?" on number of curves + nameability score together. This tells us, holding nameability fixed, whether more curves still change accuracy, and holding curves fixed, whether nameability still changes accuracy.

To evaluate:

- **Main slope:** whole range below 0 -> "busy clutter" idea wins. Whole range above 0 -> "names" idea wins. Range includes 0 -> unclear (possibly both effects cancel out).
- **Untangling:** nameability predicts accuracy and complexity doesn't -> names are what matter. Complexity predicts accuracy and nameability doesn't -> complexity is what matters.

Known limit: across the 4 versions, complexity and nameability change together, so the main slope alone can't separate them -> that's why Stage H2.2 measures nameability directly instead of assuming it.

H3: Every kind of clutter hurts counting -> all 3 counting datasets

- Acc drop for every cluttered version, compared with clean, on the same scenes.

To evaluate:

- Whole range above 0 -> that clutter hurts. Expected, but this is the baseline effect everything else builds on.
- The scorer also gives a McNemar p-value for each version vs clean -> report it next to the interval.

H4: The model counts clutter as real targets -> Shapes, Shapes Hard, Objects

- Use "overcnt" from the scorer (how often the model counted a distractor as a real shape/object) for each version.

To evaluate:

- Prediction: overcnt is highest for lookalikes in all 3 datasets (a square with a gap gets counted as a square, a tomato gets counted as an apple).
- Compare overcnt(lookalikes) with the next-highest version -> 95% interval for the difference above 0 = supported.

H5: The model easily counts incomplete shapes as complete shapes -> Shapes, Shapes Hard

Prediction: the VLM treats an incomplete shape (a gap in the outline, or one wavy side) as the complete shape it resembles. A square with a gap gets counted as a square.

Stage H5.1: Find what each incomplete shape imitates

- Use the lookalikes version of each scene. From the "objects" column, find which named shape each incomplete shape imitates (square with a gap -> square).
- Check in the trial run that the "objects" column records this for every lookalike. If it doesn't, ask Mohid before the main run.
- Check the Shapes question: if it doesn't tell the model to skip broken shapes, counting them is arguably correct there, so H5 then uses Shapes Hard only (its question says so explicitly).

Stage H5.2: Absorption rate

- For each image and each shape type: extra = model's count - true count, kept between 0 and the number of incomplete shapes of that type in the image.
- Absorption rate = total extra / total incomplete shapes -> the share of incomplete shapes that the model counted as complete ones (0 = never, 1 = always).

```
absorption rate = sum( extra ) / number of incomplete shapes
```

Stage H5.3: Compare with unnamed clutter

- Squiggles and blobs add stuff to the picture too, so the model might over-count just because the picture is busier. Measure the same extra counts in the squiggles and blobs versions, per distractor.
- Difference = absorption rate(lookalikes) - extra counts per distractor(squiggles and blobs).

Stage H5.4: Also report

- Absorption rate split by defect type: gap in the outline vs one wavy side.
- Absorption rate split by shape: circle, triangle, square, star.

To evaluate:

- Supported if: the difference in Stage H5.3 has its whole 95% range above 0 (incomplete shapes are counted more than plain clutter) AND the absorption rate's whole range is above 0.5 (the model counts most incomplete shapes as complete -> "easily"). The 0.5 cut-off is fixed before the data.
- Difference above 0 but absorption rate not clearly above 0.5 -> partly supported (the model is fooled, but not most of the time).
- Difference range includes 0 -> incomplete shapes fool the model no more than plain clutter (not supported).

H6: Text in the image fools the model -> Shapes Hard, Objects

- Acc drop(text_traps) in both datasets. The text is always wrong ("star: 4", "3 dogs").

To evaluate:

- Whole range above 0 -> the model reads the text and gets fooled.
- Extra check: how often the model's answer copies the number written in the image (the written words are in the "objects" column).

H7: Tangled lines are harder to follow -> Lines dataset

Stage H7.1: Plain clutter

- Acc drop(crossings_0) (other lines present, but none crosses the target).

Stage H7.2: Entanglement

- Slope of accuracy as crossings go 2 -> 4 -> 8.
- **Main view:** only the "rank changed" images (where guessing "the line goes straight across" gives the wrong box). In about 70% of crossing images, and all of crossings_0, guessing straight across is right, so a model can score well without actually tracing the line.

Stage H7.3: Also report

- "jumped" (how often the model switched to the other line at a crossing) for each version.
- Accuracy for "rank kept" vs "rank changed" -> a big gap means the model is guessing, not tracing.

> **To evaluate:** slope range below 0 -> more crossings = harder (supported).

H8: Colour gives a line an identity, like a name -> Lines dataset

```
Acc(crossings_4_colored) - Acc(crossings_4)
```

- Same lines, only the colour of the other lines differs (the target stays black).

> **To evaluate:** whole range above 0 -> giving the other lines a separate colour helps the model keep track of the target. This links back to the main idea: an identity (a name or a colour) helps the model keep things apart.

Always reported (not used for verdicts)

Every version of every dataset, average count error (abs_err), parse rate, and how the model's answers are spread.